SLF4J LOGGING FRAMEWORK

Exercise 1: Logging Error Messages and Warning Levels

Objective:

To use SLF4J with Logback to log messages at different severity levels (TRACE, DEBUG, INFO, WARN, ERROR) and observe how log level configuration filters which messages are displayed.

Code:

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class LoggingDemo {

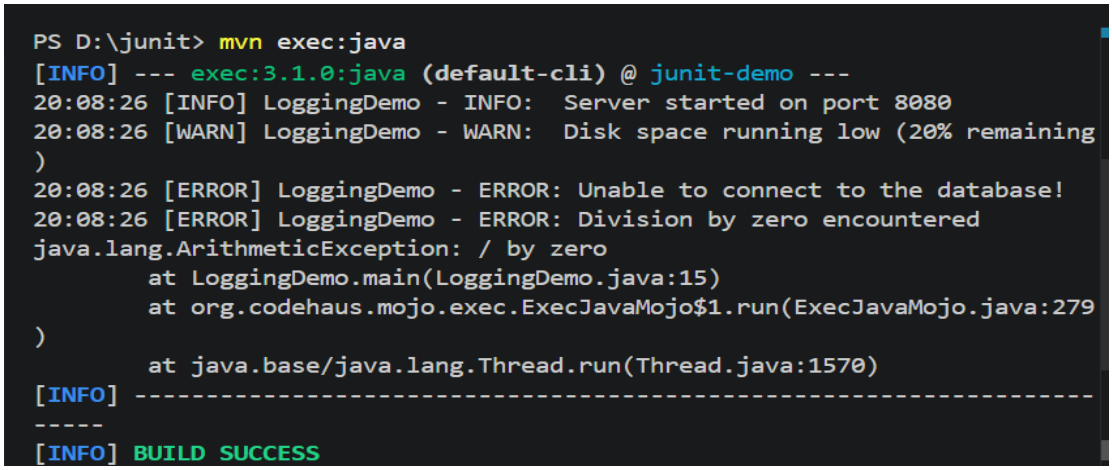
```
private static final Logger logger =
        LoggerFactory.getLogger(LoggingDemo.class);

public static void main(String[] args) {

    logger.trace("TRACE: Entering main method");
    logger.debug("DEBUG: Application starting up");
    logger.info("INFO: Server started on port 8080");
    logger.warn("WARN: Disk space running low (20% remaining)");
    logger.error("ERROR: Unable to connect to the database!");

    // Simulating a caught exception logged at ERROR level
    try {
        int result = 10 / 0;
    } catch (ArithmeticException e) {
        logger.error("ERROR: Division by zero encountered", e);
    }
}
```

}

logback.xml (configuration):

```
<!-- Change level to DEBUG or TRACE to see more output -->
<root level="INFO">
    <appender-ref ref="CONSOLE"/>
</root>
```

Program Output Screenshot:



Output Text:

With root log level set to INFO, only INFO, WARN, and ERROR messages appear. TRACE and DEBUG messages are suppressed. The ArithmeticException stack trace is included with the ERROR log automatically.

Result:

SLF4J successfully logged messages at multiple levels. The Logback configuration controlled which levels were visible at runtime without changing application code.

Conclusion:

SLF4J provides a simple, unified logging API that is decoupled from the underlying logging framework (Logback, Log4j, etc.). Log levels (TRACE < DEBUG < INFO < WARN < ERROR) allow developers to control verbosity through configuration alone, making it easy to switch between verbose development logs and concise production logs.